

Thonny + Raspberry Pi Pico

Écrire son code, l'enregistrer et le lancer sur le Pico !

Idée clé

- **Thonny** est un environnement de développement (IDE) simple et adapté à Python.
- On écrit le code **dans l'éditeur**, on l'enregistre sur l'ordinateur, puis on l'envoie sur le Raspberry Pi Pico.
- Le Pico exécute le code et on voit les résultats dans la console.

1. Installer et lancer Thonny

1. Télécharge Thonny sur le site officiel : https://thonny.org
2. Installe le programme.
3. Ouvre Thonny.

Thonny
Python IDE

Bon à savoir

- Thonny est gratuit, léger et fonctionne sur Windows, Mac et Linux.
- Il est déjà configuré pour travailler avec le Raspberry Pi Pico.

2. Connecter le Pico

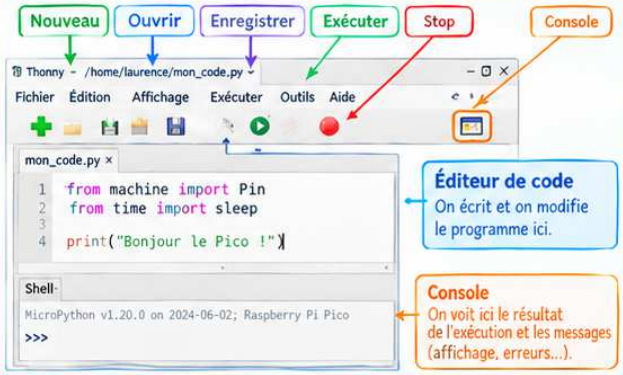


1. Branche le Pico à l'ordinateur avec un câble USB.
2. Le **Pico** apparaît comme un disque (RPI-RP2) dans l'explorateur de fichiers.
3. Dans Thonny, vérifie que l'interpréteur est bien réglé sur **MicroPython** (Raspberry Pi Pico).

Vérification rapide

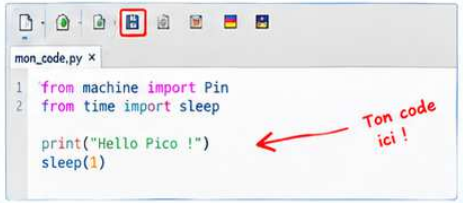
- En bas à droite de Thonny : « **MicroPython** (Raspberry Pi Pico) »
- Si ce n'est pas le cas : clique sur la petite flèche et choisis « **MicroPython** (Raspberry Pi Pico) ».

3. L'interface de Thonny



4. Écrire et enregistrer son code

1. Écris ton code dans l'éditeur.
2. Enregistre-le avec l'icône 💾 ou Ctrl+S.
3. Donne-lui un nom (ex : mon_code.py).



5. Exécuter le code sur le Pico

1. Clique sur le bouton ▶ ou appuie sur **F5**.
2. Thonny envoie le code sur le **Pico**.
3. Le résultat s'affiche dans la console en bas.

Raccourcis clavier utiles

F5	Exécuter le script actuel
Ctrl + S	Enregistrer
Ctrl + O	Ouvrir un fichier
Ctrl + N	Nouveau fichier



6. Gérer les fichiers et le programme

- Tu peux créer plusieurs fichiers pour organiser ton code.
- Les fichiers sont enregistrés sur ton ordinateur, dans le dossier où tu les as sauvegardés.
- Pour ouvrir un fichier existant : icône 📂 ou **Ctrl+O**.
- Pour créer un nouveau fichier : icône 📄 ou **Ctrl+N**.

Astuce

Le fichier principal s'appelle souvent `main.py` (mais tu peux lui donner le nom que tu veux).



À retenir

- ✔ Thonny est un IDE simple et efficace.
- ✔ On écrit, on enregistre, on exécute.
- ✔ Le code est envoyé sur le Pico via USB.
- ✔ On voit les résultats dans la console.
- ✔ Tu peux utiliser plusieurs fichiers.
- ✔ Les raccourcis clavier font gagner du temps.

Et maintenant... à toi de jouer !



La LED

Une petite diode qui fait de la lumière

1. Qu'est-ce qu'une LED ?

Une LED (Light Emitting Diode) est une diode qui émet de la lumière lorsqu'elle est parcourue par un courant électrique.

- Elle consomme peu d'énergie.
- Elle existe en plusieurs couleurs (rouge, verte, bleue, blanche, etc.).
- Elle ne fonctionne que dans un seul sens (comme une diode classique).

2. Les bornes : anode et cathode



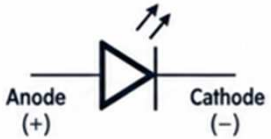
La LED possède deux pattes :

- **Anode (+) :** la patte la plus longue.
- **Cathode (–) :** la patte la plus courte.

Anode (+) (patte longue)

Cathode (–) (patte courte)

3. Symbole et sens du courant



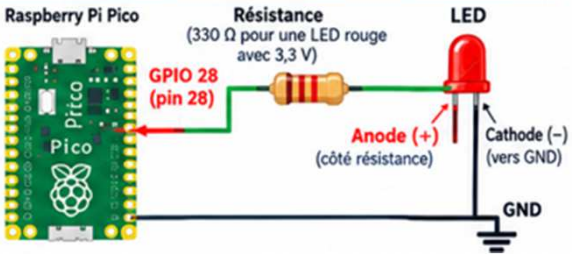
Symbole d'une LED

Le courant conventionnel va de l'anode (+) vers la cathode (–).

4. Valeurs de résistance conseillées

Couleur de la LED	Tension de seuil (approx.)	Résistance (pour 3,3 V)	Résistance (pour 5 V)
Rouge	1,8 – 2,2 V	330 Ω	220 Ω
Verte	2,0 – 2,4 V	330 Ω	220 Ω
Jaune	2,0 – 2,4 V	330 Ω	220 Ω
Bleue	3,0 – 3,4 V	150 Ω	100 Ω
Blanche	3,0 – 3,4 V	150 Ω	100 Ω

5. Montage avec une résistance



6. Exemple de code en MicroPython

```
from machine import Pin
from time import sleep

# LED sur la broche 28 (GP28)
LED = Pin(28, Pin.OUT)

# 1. On allume la LED pendant 1 seconde
#    puis on l'éteint pendant 1 seconde
LED.value(1)     # LED allumée
sleep(1)         # on attend 1 seconde
LED.value(0)     # LED éteinte
sleep(1)         # on attend 1 seconde

# 2. On utilise la fonction toggle qui
#    inverse l'état de la LED
#    (éteinte → allumée, allumée → éteinte)
LED.toggle()     # LED allumée
sleep(1)         # on attend 1 seconde
LED.toggle()     # LED éteinte
sleep(1)         # on attend 1 seconde

# 3. On la fait clignoter pour toujours
while True:
    LED.toggle()     # change l'état de la LED
    sleep(1)         # on attend 1 seconde
```



Le bouton poussoir

Une petite pression, un grand effet

1. Qu'est-ce qu'un bouton pousoir ?

Un bouton poussoir est un interrupteur qui ferme le circuit uniquement pendant la pression.

2. Les bornes :

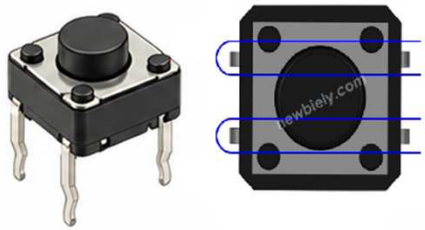
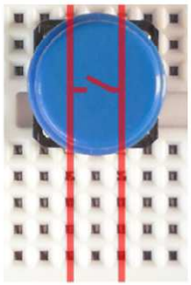


Schéma électrique interne :



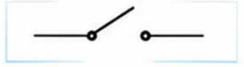
3. Symbole et principe



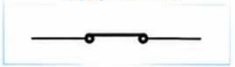
Symbole d'un bouton poussoir (NO)

Quand on appuie, le contact se ferme et le courant peut passer.

Relâché (ouvert)

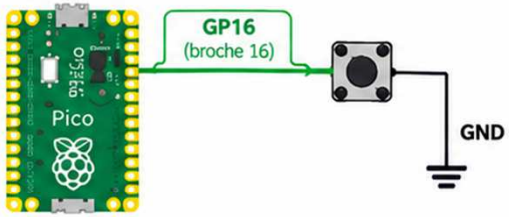


Appuyé (fermé)



4. Montage avec le Raspberry Pi Pico

Raspberry Pi Pico



Le bouton est connecté entre la broche16 (GP16) et la masse (GND).

Élément	Connexion
Bouton poussoir	Entre GP16 et GND
Résistance de pull-up (interne)	Activée dans le code
Alimentation	Pas nécessaire (utilisation de la pull-up interne)

5. Exemple de code en MicroPython

```
from machine import Pin
from time import sleep_ms

bouton = Pin(16, machine.Pin.IN, Pin.PULL_UP)

while True:
    if bouton.value() == 0:
        print("Bouton Appuyé")
    else:
        print("Bouton Relaché")
    sleep_ms(50)
```

Comment ça marche ?

- Pin(16, machine.Pin.IN, Pin.PULL_UP) active la résistance de pull-up interne.
- Quand on appuie : la broche passe à 0 (niveau bas).
- Quand on relâche : la broche reste à 1 (niveau haut).
- sleep_ms(50) évite une lecture trop rapide (anti-rebond).

À retenir

- Le bouton poussoir est un interrupteur momentané.
- Il ne reste actif que lorsqu'on appuie dessus.
- Ici, on utilise la résistance de pull-up interne du Pico.
- Quand on appuie : l'entrée passe à 0 (niveau bas).
- Quand on relâche : l'entrée reste à 1 (niveau haut).



MicroPython – Notions avancées

Le bouton poussoir : interruptions, fonctions et variables

Objectif

Utiliser un bouton poussoir avec des interruptions pour réagir immédiatement à un appui, en évitant les rebonds, et en organisant le code avec des fonctions, des variables et des fonctions de temps.

1. Import time

Le module `time` permet de gérer le temps.

- `time.sleep(x)` : pause en secondes (ex : 1)
- `time.sleep_ms(x)` : pause en millisecondes (ex : 50)
- `time.ticks_ms()` : temps écoulé en ms depuis le démarrage
- `time.ticks_diff(t1, t0)` : différence entre deux temps (en ms)

`ticks_ms()` et `ticks_diff()` sont très utiles pour mesurer des délais précis et gérer les rebonds.

2. Montage pull up / pull down

Le bouton poussoir peut être câblé avec une résistance pull-up (utilisée ici) ou pull-down.

Pull-up (interne)

- Au repos : **niveau haut (1)**
- Appui : niveau bas (0)

Pull-down (externe)

- Au repos : niveau bas (0)
- Appui : niveau haut (1)

3. Interruptions (IRQ)

Une interruption permet d'exécuter une fonction dès qu'un événement se produit (ex : appui sur le bouton).

```
bouton.irq(trigger=Pin.IRQ_RISING,
           handler=bouton_handler)
```

Les déclencheurs possibles :

- `Pin.IRQ_RISING` : front montant (appui)
- `Pin.IRQ_FALLING` : front descendant (relâchement)
- `Pin.IRQ_RISING_FALLING` : les deux

Les interruptions sont très rapides :
il faut donc éviter les traitements longs dans la fonction d'interruption (ici on fait juste une vérification et on met une variable).

4. Fonctions

Une fonction regroupe un bloc de code réutilisable.

```
def ma_fonction(parametres):
    # instructions
    return valeur
```

- **def** : on définit une fonction
- Les paramètres permettent de passer des informations
- **return** permet de renvoyer un résultat (optionnel)

5. Variables & variables globales

Une variable sert à stocker une valeur. Une variable globale est accessible partout dans le programme.

Variable locale

- utilisée uniquement dans la fonction
- pas besoin de global

Variable globale

- déclarée en dehors des fonctions
- pour la modifier dans une fonction : il faut utiliser `global`

Exemple :
la variable `bouton_actif` est une variable globale, modifiée dans la fonction d'interruption et lue dans la fonction `test_bouton`.

Code complet : exemple

```
from machine import Pin
import time

# Configuration du bouton : entrée avec pull-up interne
bouton = Pin(16, Pin.IN, Pin.PULL_UP)

# Variables globales
date_dernier_appui = time.ticks_ms()
bouton_actif = False

# Fonction appelée lors de l'appui sur le bouton
def bouton_handler(pin):
    global bouton_actif
    global date_dernier_appui
    # On évite les rebonds (délai de 500 ms)
    if time.ticks_diff(time.ticks_ms(), date_dernier_appui) > 500:
        bouton_actif = True
        # On réinitialise la variable date_dernier_appui
        date_dernier_appui = time.ticks_ms()


# Fonction qui associe l'appui sur le bouton à la fonction ci-dessus
bouton.irq(trigger=Pin.IRQ_RISING, handler=bouton_handler)

# Fonction de test : affiche l'état du bouton
def test_bouton():
    global bouton_actif
    # On attend 1 ms
    time.sleep(1)
    # On regarde l'état du bouton
    if bouton_actif:
        print("Bouton actif")
        bouton_actif = False

# Programme principal
print("En attente d'un appui sur le BP")
while True:
    test_bouton()
```

Dans ce code, on utilise :

- `time.sleep` / `time.ticks_ms` / `time.ticks_diff`
- `Pin.IN` / `Pin.PULL_UP`
- IRQ (interruption) + une fonction
- des fonctions, des variables globales et locales

À retenir

- ✓ **time** : sleep, sleep_ms, ticks_ms, ticks_diff.
- ✓ **Pull-up / Pull-down** : montage et configuration
- ✓ **Interruptions** : IRQ_RISING, handler
- ✓ **Fonctions** : def, paramètres, return
- ✓ **Variables** : locales et globales (global)
- ✓ **Éviter les rebonds** : délai avec ticks_diff



Montages Pull-Up / Pull-Down

Forcer un état connu sur une entrée GPIO

À retenir

- Une entrée GPIO laissée seule est **flottante** : elle peut lire 0 ou 1 de manière imprévisible.
- Le pull-up/pull-down est une résistance qui fixe l'état de l'entrée au repos.
- Le bouton relie l'entrée à la masse (pull-up) ou au +3,3 V (pull-down) lorsqu'on appuie.

1. Entrée flottante (sans résistance)

Une entrée GPIO non connectée ou laissée "en l'air" peut lire 0 ou 1 de façon aléatoire (bruit, parasites…).

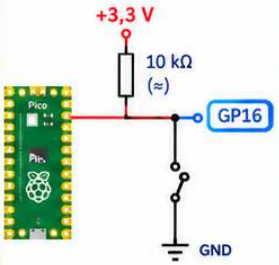
Il faut donc la forcer avec une résistance pull-up ou pull-down.

Sans résistance : état indéterminé !

0 ? 1 ?

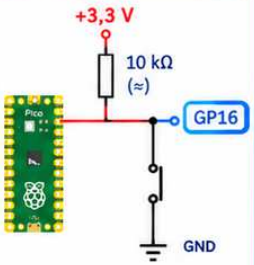
2. Pull-Up (résistance vers +3,3 V)

Relâché (bouton non appuyé)



L'entrée est reliée à +3,3 V via la résistance → **logique 1**

Appuyé (bouton pressé)

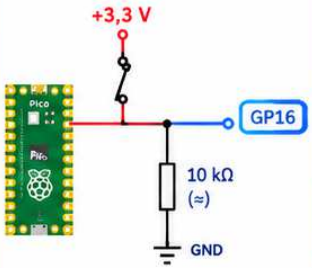


Le bouton relie l'entrée à GND → **logique 0**

Pull-up : repos = 1 | appui = 0

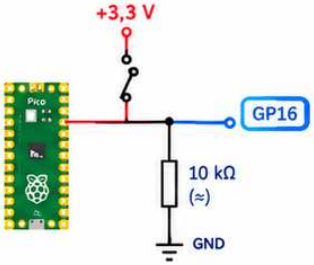
3. Pull-Down (résistance vers GND)

Relâché (bouton non appuyé)



L'**entrée** est reliée à GND via la résistance → **logique 0**

Appuyé (bouton pressé)



Le bouton relie l'entrée à +3,3 V → **logique 1**

Pull-down : repos = 0 | appui = 1

4. Avec le Raspberry Pi Pico

- Les résistances pull-up et pull-down peuvent être internes au Pico (pas besoin de les ajouter sur le montage).
- On les active dans le code avec :

```
Pin.PULL_UP  ou  Pin.PULL_DOWN
```

- Pour notre bouton :

```
bouton = Pin(16, Pin.IN, Pin.PULL_UP)
```

- Le bouton est connecté entre GP16 et GND.

5. Tableau récapitulatif

Montage	Résistance	au repos (aucun appui)	à l'appui (bouton pressé)
Pull-up	vers 3,3 V (+3,3 V)	1 (haut)	0 (bas)
Pull-down	vers GND (masse)	0 (bas)	1 (haut)

En résumé

- Pull-up : résistance vers +3,3 V → repos = 1, appui = 0
- Pull-down : résistance vers GND → repos = 0, appui = 1
- Sur le Pico : résistances internes activées avec `Pin.PULL_UP` / `Pin.PULL_DOWN`.
- Notre montage : bouton entre GP16 et GND, avec `Pin(16, Pin.IN, Pin.PULL_UP)`.